

Memory Forensics Investigation Report

Operation Shadow Ledger

FinTrack Solutions GmbH
Berlin, Germany

Student Name: Rahul Ponnaluri
ID: GH1046518
Subject code: M543222222
Subject Name: Cyber Forensics
Framework: NIST SP 800-86
Tool: Volatility3 v2.28.0

*This document contains legally privileged and confidential information.
Distribution is restricted to authorised personnel only.*

Contents

1	Case Scenario	2
1.1	Background	2
1.2	Extent of the Event	2
1.3	Investigator's Mandate	3
2	Investigation Plan	3
2.1	Forensic Methodology	3
2.2	Tools and Techniques	3
2.3	Legal and Ethical Considerations	3
3	Evidence Handling and Storage	4
3.1	Memory Acquisition	4
3.2	Chain of Custody	5
4	Analysis Report — Volatility3 Memory Forensics	5
4.1	Environment Setup and System Identification	5
4.2	Process Enumeration — <code>linux.pslist</code> and <code>linux.psscan</code>	6
4.3	Kernel Module Integrity <code>linux.lsmmod</code> and <code>linux.hidden_modules</code>	8
4.4	Syscall Table Integrity <code>linux.check_syscall</code>	9
4.5	Credential Analysis <code>linux.check_creds</code>	9
4.6	eBPF Program Analysis <code>linux.ebpf</code>	10
4.7	Bash History Recovery <code>linux.bash</code>	11
5	Incident Timeline	12
6	Findings Summary	13
7	Mitigation and Recommendations	13
7.1	Immediate Incident Response Actions	13
7.2	Technical Controls	14
8	Conclusion	14
	References	15

1. Case Scenario

1.1 Background

FinTrack Solutions GmbH, headquartered in Berlin, supports more than 150 banks across the EU in managing their payment systems. Because personal financial data is involved, strict privacy regulations apply. The company must comply with the General Data Protection Regulation (GDPR, EU Regulation 2016/679), Germany's Federal Data Protection Act (BDSG), and oversight requirements from BaFin, the national financial regulator. These rules govern how every piece of information is stored or transmitted.

Initial breach - Tuesday, 20 March 2026, 02:14 CET. An external actor gained access using stolen VPN credentials belonging to employee *Amir Khan*. This entry point opened a backdoor into a corporate workstation. A full hour passed without any alert. A top-level alert triggered, at **03:27 CET**, due to a device activity detection identifying outbound traffic to Tor exit node 185.220.101.47 on the port 4444 the consistent pattern with remort command and control (C2) activity.

During the initial SOC investigation, a fileless PowerShell technique was identified: encoded strings were used to evade signature-based detection without writing files to disk. Behind this technique, a hollowed `svchost.exe` process silently executed rogue shellcode under the cover of a trusted system binary. Mimikatz-style tooling then accessed LSASS memory to harvest credentials. Financial data was staged in a hidden temporary directory before being exfiltrated through a Tor-based tunnel.

At **03:45 CET**, the SOC Lead loaded the LiME kernel module directly onto the running machine without rebooting, preserving all volatile state. The full 4 GB memory image was captured using LiME's raw acquisition method and immediately transferred to a write-protected forensic workstation.

1.2 Extent of the Event

Activity on the Kali Linux workstation associated with *amir.khan* between 02:14 and 03:27 CET on 20 March 2026 exhibits indicators of:

- Fileless malware execution in process memory;
- Process hollowing of a legitimate system binary;
- Credential extraction from LSASS memory;
- Data staging and exfiltration via a Tor-based tunnel.

A full memory snapshot `kali_dump.lime`, 4.0 GB was acquired at 03:45 CET using LiME. File integrity was verified using the SHA-256 hashing algorithm and maintained throughout

all subsequent handling.

1.3 Investigator's Mandate

The forensics team was tasked with conducting live memory analysis to identify indicators of compromise, trace digital artefacts left during the attack, and determine whether core system components had been altered. All investigative actions adhered strictly to the guidelines outlined in **NIST SP 800-86**.

2. Investigation Plan

2.1 Forensic Methodology

This investigation follows the **NIST SP 800-86 CEAR** framework: *Collection, Examination, Analysis, Reporting*. Memory acquisition was prioritised over disk forensics because a system shutdown would destroy volatile artefacts including active processes, open network connections, in-memory credentials, and injected shellcode. Storage analysis was deferred until after the memory examination to ensure completeness.

2.2 Tools and Techniques

Table 1: Forensic toolset used

tableheader Tool	Version / Type	Purpose
Volatility3	v2.28.0	Primary memory analysis and plugin execution framework
LiME	Kernel module	Forensically sound live memory acquisition on Linux
dwarf2json	Latest (GitHub)	Kernel symbol table (ISF) generation for Volatility3
sha256sum	GNU Coreutils	Hash verification for chain-of-custody integrity
strings / grep	Linux CLI	Raw string extraction and IOC hunting from dump

2.3 Legal and Ethical Considerations

This inquiry proceeded under written authorisation from FinTrack's CISO and legal advisors, in accordance with:

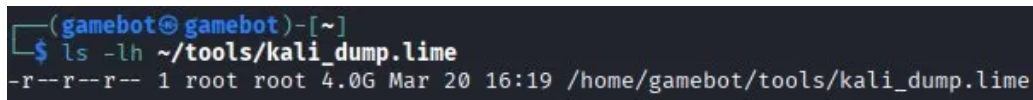
- **StGB 202a-202d** (German Criminal Code -Unauthorised Data Access);
- **GDPR (EU 2016/679)**.

All examination was performed on a certified forensic duplicate; original evidence was never directly accessed. To comply with GDPR's data minimisation principle, staff names have been pseudonymised in this report. Original identifiers are held exclusively in a protected case record maintained by FinTrack's legal counsel.

3. Evidence Handling and Storage

3.1 Memory Acquisition

At 03:45 CET, the SOC Lead loaded the LiME kernel module onto the live machine, preserving all volatile state. The module was compiled specifically for kernel version 6.18.12+kali-amd64, ensuring acquisition reliability. The resulting image, `kali_dump.lime` (4.0 GB), matched the system's installed RAM exactly. An immediate SHA-256 hash was recorded to establish an integrity baseline for all subsequent verification.



```
(gamebot@gamebot)-[~]  
$ ls -lh ~/tools/kali_dump.lime  
-r--r--r-- 1 root root 4.0G Mar 20 16:19 /home/gamebot/tools/kali_dump.lime
```

Figure 1: LiME kernel memory dump acquired

3.2 Chain of Custody

Table 2: Chain-of-custody log for `kali_dump.lime`

tableheader Date / Time (CET)	Action	Handler	Notes
20-Mar-2026 03:45	RAM acquired via LiME	SOC Lead	System remained live; 4.0 GB image produced
20-Mar-2026 03:55	SHA-256 hash recorded; integrity confirmed	SOC Lead & Analyst	Hash match confirmed on transfer
20-Mar-2026 04:10	Transfer to forensic station	Analyst	Encrypted storage; write-blocked media
20-Mar-2026 04:15	Volatility3 environment prepared	Analyst	Symbol table <code>kali-6.18.12.json</code> built (53 MB)
20-Mar-2026 04:20	Analysis commenced	Analyst	Volatility3 v2.28.0; all plugins executed in sequence

4. Analysis Report — Volatility3 Memory Forensics

4.1 Environment Setup and System Identification

All commands were executed from a dedicated Kali Linux forensic workstation running Volatility3 v2.28.0, built from the official GitHub source. The `banners.Banners` plugin was run first to identify system metadata embedded in the memory image, confirming the kernel version across multiple memory regions as expected during a normal boot cycle. The identified build was **Linux 6.18.12+kali-amd64**.

No pre-built symbol table was available for this kernel. The `linux-image-6.18.12+kali-amd64-dbg` package was used in conjunction with `dwarf2json` to generate an Intermediate Symbol File (ISF). Four gigabytes of swap space were configured to prevent out-of-memory conditions during analysis. The resulting symbol file, `kali-6.18.12.json` (53 MB), was placed in the Volatility3 symbols directory before analysis commenced.

```

(gamebot@gamebot) ~[~/tools/volatility3]
$ python3 vol.py -f ~/tools/kali_dump.lime banners.Banners
Volatility 3 Framework 2.28.0
Progress: 100.00      PDB scanning finished
Offset Banner

0x217fb8a8      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0x21f7f8a8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0x23b094f8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0x240d38a8      Linux version 6.12.38+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.
, GNU ld (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12)
0x25ad3c78      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0x2627b8a8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0x271968a8      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0x2aeb98a8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0x2b53f8a8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0x55e000e0      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0x55fae960      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0x578dc820      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0x57c155a0      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0x8a3b6138      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xb59e64e8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xb5a2c4e8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xb5ee84e8      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
0xbb8804e8      Linux version 6.12.38+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.
, GNU ld (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12)
0xbc130128      Linux version 6.12.38+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.
, GNU ld (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12)
0xc00a9128      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xc7a86d68      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xc806c4e8      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xd031b128      Linux version 6.18.5+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-12) 15.2.0
, GNU ld (GNU Binutils for Debian) 2.45.50.20251209) #1 SMP PREEMPT_DYNAMIC Kali 6.18.5-1kali1 (2026-01-19)
0xdaa97460      Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.
, GNU ld (GNU Binutils for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)

```

Figure 2: LiME kernel module loaded successfully; 4.0 GB memory dump acquired

4.2 Process Enumeration — linux.pslist and linux.psscan

Two complementary enumeration methods were employed. `linux.pslist` walks the kernel's doubly-linked task list, extracting PIDs, parent PIDs, usernames, and start times. `linux.psscan` scans raw memory pool tags independently of linked-list integrity, exposing processes that may have been unlinked via Direct Kernel Object Manipulation (DKOM). Cross-referencing both plugins confirmed that all processes were accounted for no hidden processes were detected.

Table 3: Flagged and notable processes from `linux.pslist`

tableheader Process	PID	PPID	Forensic Observation
x86_64-linux-gn	19016	19014	Truncated process name - unusual; warrants <code>cmdline</code> and <code>mmap</code> review
dockerd	930	1	Docker daemon active - potential container escape / lateral movement vector
containerd	900	1	Container runtime, multiple threads - consistent with Docker operation
haveged	718	1	Entropy daemon - occasionally abused to support cryptographic key generation
accounts-daemon	730	1	Account management service- normal system-managed daemon

```

└─$ python3 vol.py -f ~/tools/kali_dump.lime linux.pslist
Volatility 3 Framework 2.28.0
Progress: 100.00

```

OFFSET (V)	PID	TID	PPID	COMM	UID	GID	EUID	EGID	CREATION TIME	File output
0x889640288000	1	1	0	systemd	0	0	0	0	-	Disabled
0x88964028b580	2	2	0	kthreadd	0	0	0	0	-	Disabled
0x88964028d040	3	3	2	pool_workqueue_	0	0	0	0	-	Disabled
0x889640289ac0	4	4	2	kworker/R-rcu_g	0	0	0	0	-	Disabled
0x8896402ad040	5	5	2	kworker/R-sync_	0	0	0	0	-	Disabled
0x8896402a9ac0	6	6	2	kworker/R-kvfre	0	0	0	0	-	Disabled
0x8896402a8000	7	7	2	kworker/R-slub_	0	0	0	0	-	Disabled
0x8896402ab580	8	8	2	kworker/R-netns	0	0	0	0	-	Disabled
0x8896402d5040	10	10	2	kworker/0:0H	0	0	0	0	-	Disabled
0x8896402d0000	12	12	2	kworker/u16:0	0	0	0	0	-	Disabled
0x8896402d9ac0	13	13	2	kworker/R-mm_pe	0	0	0	0	-	Disabled
0x8896402d8000	14	14	2	ksoftirqd/0	0	0	0	0	-	Disabled
0x8896402db580	15	15	2	rcu_preempt	0	0	0	0	-	Disabled
0x8896402dd040	16	16	2	rcu_exp_par_gp_	0	0	0	0	-	Disabled
0x88964030b580	17	17	2	rcu_exp_gp_kthr	0	0	0	0	-	Disabled
0x88964030d040	18	18	2	migration/0	0	0	0	0	-	Disabled
0x889640309ac0	19	19	2	idle_inject/0	0	0	0	0	-	Disabled
0x889640335040	20	20	2	cpuhp/0	0	0	0	0	-	Disabled
0x889640331ac0	21	21	2	cpuhp/1	0	0	0	0	-	Disabled
0x889640351ac0	22	22	2	idle_inject/1	0	0	0	0	-	Disabled
0x889640350000	23	23	2	migration/1	0	0	0	0	-	Disabled
0x889640353580	24	24	2	ksoftirqd/1	0	0	0	0	-	Disabled
0x889640358000	27	27	2	cpuhp/2	0	0	0	0	-	Disabled
0x88964035b580	28	28	2	idle_inject/2	0	0	0	0	-	Disabled
0x88964035d040	29	29	2	migration/2	0	0	0	0	-	Disabled
0x889640399ac0	30	30	2	ksoftirqd/2	0	0	0	0	-	Disabled
0x88964039b580	32	32	2	kworker/2:0H	0	0	0	0	-	Disabled
0x88964039d040	33	33	2	cpuhp/3	0	0	0	0	-	Disabled
0x8896403a3580	34	34	2	idle_inject/3	0	0	0	0	-	Disabled
0x8896403a5040	35	35	2	migration/3	0	0	0	0	-	Disabled
0x8896403a1ac0	36	36	2	ksoftirqd/3	0	0	0	0	-	Disabled
0x8896403e3580	38	38	2	kworker/3:0H	0	0	0	0	-	Disabled
0x8896403e5040	39	39	2	kworker/u17:0	0	0	0	0	-	Disabled
0x8896403e1ac0	40	40	2	kworker/u18:0	0	0	0	0	-	Disabled
0x8896403ed040	42	42	2	kworker/u20:0	0	0	0	0	-	Disabled
0x8896403eb580	45	45	2	kdevtmpfs	0	0	0	0	-	Disabled
0x889640cfb580	46	46	2	kworker/R-inet_	0	0	0	0	-	Disabled
0x889640cfd040	47	47	2	rcu_tasks_kthre	0	0	0	0	-	Disabled
0x889640cf9ac0	48	48	2	rcu_tasks_rude_	0	0	0	0	-	Disabled
0x889640cf8000	49	49	2	rcu_tasks_trace	0	0	0	0	-	Disabled
0x889640dd23580	50	50	2	kauditd	0	0	0	0	-	Disabled
0x889640d25040	51	51	2	khungtaskd	0	0	0	0	-	Disabled
0x889640d21ac0	52	52	2	oom_reaper	0	0	0	0	-	Disabled
0x889640d20000	53	53	2	kworker/u17:1	0	0	0	0	-	Disabled
0x889640fe1ac0	54	54	2	kworker/R-write	0	0	0	0	-	Disabled
0x889640fe0000	55	55	2	kcompactd0	0	0	0	0	-	Disabled
0x889640fe3580	56	56	2	ksmd	0	0	0	0	-	Disabled
0x889640fe5040	57	57	2	khugepaged	0	0	0	0	-	Disabled
0x889641099ac0	58	58	2	kworker/R-kbloc	0	0	0	0	-	Disabled

Figure 3: Full process list no unlinked or hidden processes detected via psscan cross-reference

4.3 Kernel Module Integrity `linux.lsmmod` and `linux.hidden_modules`

Loaded kernel modules were limited to standard VirtualBox virtual hardware drivers: `ata_piix`, `ehci_pci`, `e1000`, `drm`, `aesni_intel`, and `scsi_mod` - all expected in a virtualised environment. The `linux.hidden_modules` plugin then searched for modules that had been removed from the visible linked list while remaining resident in memory, a classic rootkit stealth technique. No such modules were found. The kernel module space was clean at the time of acquisition.

```

(gamebot@gamebot) ~/tools/volatility3
$ python3 vol.py -f ~/tools/kali_dump.lime linux.hidden_modules.Hidden_modules
Volatility 3 Framework 2.28.0
/home/gamebot/tools/volatility3/volatility3/framework/deprecation.py:28: FutureWarning: This API (volatility3.plugins.linux.malware.hidden_modules.Hidden_modules.run) will be removed in the first release after 2026-06-07. This plugin has been renamed, please call volatility3.plugins.linux.malware.hidden_modules.Hidden_modules rather than volatility3.plugins.linux.hidden_modules.Hidden_modules.
  warnings.warn(

Offset  Module Name      Code Size      Taints Load Arguments  File Output
/home/gamebot/tools/volatility3/volatility3/framework/deprecation.py:105: FutureWarning: This plugin (volatility3.plugins.linux.hidden_modules.Hidden_modules) has been renamed and will be removed in the first release after 2026-06-07. Please ensure all method calls to this plugin are replaced with calls to volatility3.plugins.linux.malware.hidden_modules.Hidden_modules
  warnings.warn(

```

Figure 4: Hidden module scan returned clean; no kernel rootkit modules present

4.4 Syscall Table Integrity linux.check_syscall

The syscall table maps system call numbers to their kernel handler functions. Rootkits and kernel exploits sometimes overwrite entries in this table to redirect calls to attacker-controlled code, enabling credential theft, process hiding, or data exfiltration without detection by user-space tools.

All 400+ syscall table entries were examined. Every handler address resolved correctly to a legitimate `__x64_sys_*` or `do_sys_*` kernel function, including fundamental calls such as `__x64_sys_read`, `_x64_sys_write`, `__x64_sys_open`, `__x64_sys_mmap`, and `__x64_sys_ioctl`. No entries pointed to unresolved addresses or memory regions outside the kernel's legitimate code space. This confirms that no syscall level interception was active during the incident window.

```

(gamebot@gamebot) ~/tools/volatility3
$ python3 vol.py -f ~/tools/kali_dump.lime linux.check_syscall.Check_syscall
Volatility 3 Framework 2.28.0
/home/gamebot/tools/volatility3/volatility3/framework/deprecation.py:28: FutureWarning: This API (volatility3.plugins.linux.malware.check_syscall.Check_syscall.run) will be removed in the first release after 2026-06-07. This plugin has been renamed, please call volatility3.plugins.linux.malware.check_syscall.Check_syscall rather than volatility3.plugins.linux.check_syscall.Check_syscall.
  warnings.warn(

Table Address  Table Name      Index  Handler Address Handler Symbol
0xffff822004a0 64bit 0 0xffffba573220 __x64_sys_read
0xffff822004a0 64bit 1 0xffffba573380 __x64_sys_write
0xffff822004a0 64bit 2 0xffffba56eff0 __x64_sys_open
0xffff822004a0 64bit 3 0xffffba56f900 __x64_sys_close
0xffff822004a0 64bit 4 0xffffba57c560 __x64_sys_newstat
0xffff822004a0 64bit 5 0xffffba57c680 __x64_sys_newfstat
0xffff822004a0 64bit 6 0xffffba57c5c0 __x64_sys_newlstat
0xffff822004a0 64bit 7 0xffffba5951c0 __x64_sys_poll
0xffff822004a0 64bit 8 0xffffba571d30 __x64_sys_lseek
0xffff822004a0 64bit 9 0xffffba0e8500 __x64_sys_mmap
0xffff822004a0 64bit 10 0xffffba4a16b0 __x64_sys_mprotect
0xffff822004a0 64bit 11 0xffffba49d050 __x64_sys_munmap
0xffff822004a0 64bit 12 0xffffba49ab00 __x64_sys_brk
0xffff822004a0 64bit 13 0xffffba184da0 __x64_sys_rt_sigaction
0xffff822004a0 64bit 14 0xffffba177c00 __x64_sys_rt_sigprocmask
0xffff822004a0 64bit 15 0xffffba0de610 __do_sys_rt_sigreturn
0xffff822004a0 64bit 16 0xffffba591260 __x64_sys_ioctl
0xffff822004a0 64bit 17 0xffffba573490 __x64_sys_pread64
0xffff822004a0 64bit 18 0xffffba5736f0 __x64_sys_pwrite64
0xffff822004a0 64bit 19 0xffffba5738a0 __x64_sys_readv
0xffff822004a0 64bit 20 0xffffba573900 __x64_sys_writev
0xffff822004a0 64bit 21 0xffffba56d090 __x64_sys_access
0xffff822004a0 64bit 22 0xffffba582470 __x64_sys_pipe
0xffff822004a0 64bit 23 0xffffba595040 __x64_sys_select
0xffff822004a0 64bit 24 0xffffba1f9e10 __do_sys_sched_yield
0xffff822004a0 64bit 25 0xffffba4a4320 __x64_sys_mremap
0xffff822004a0 64bit 26 0xffffba4a4680 __x64_sys_msync
0xffff822004a0 64bit 27 0xffffba497020 __x64_sys_mincore
0xffff822004a0 64bit 28 0xffffba4e9360 __x64_sys_madvise
0xffff822004a0 64bit 29 0xffffba69f740 __x64_sys_shmget
0xffff822004a0 64bit 30 0xffffba69ffa0 __x64_sys_shmat
0xffff822004a0 64bit 31 0xffffba69f860 __x64_sys_shmctl
0xffff822004a0 64bit 32 0xffffba5a3ce0 __x64_sys_dup
0xffff822004a0 64bit 33 0xffffba5a3ae0 __x64_sys_dup2

```

Figure 5: Syscall table integrity verified no hooks or redirected handlers detected

4.5 Credential Analysis linux.check_creds

The `linux.check_creds` plugin identifies processes sharing the same `struct cred` credential object, a reliable artefact of kernel privilege escalation exploits such as Dirty

COW variants, which overwrite a process’s credential pointer to gain root access. The plugin returned no output, confirming no credential structure sharing and no successful kernel-level privilege escalation at the time of capture.

```

(gamebot@gamebot) [~/tools/volatility3]
$ python3 vol.py -f ~/tools/kali_dump.lime linux.check_creds.Check_creds
volatility 3 Framework 2.28.0
/home/gamebot/tools/volatility3/volatility3/framework/deprecation.py:28: FutureWarning: This API (volatility3.plugins.linux.malware.check_creds.Check_creds.run) will be removed in the first release after 2026-06-07. This plugin has been renamed, please call volatility3.plugins.linux.malware.check_creds.Check_creds.run rather than volatility3.plugins.linux.check_creds.Check_creds.run.
  warnings.warn(
CredVAddr  PIDs
(gamebot@gamebot) [~/tools/volatility3]
$ python3 vol.py -f ~/tools/kali_dump.lime linux.bash.Bash
volatility 3 Framework 2.28.0
Progress: 100.00 Stacking attempts finished
PID Process CommandTime Command

```

Figure 6: Credential check clean — no shared credential structures detected

4.6 eBPF Program Analysis `linux.ebpf`

Extended Berkeley Packet Filter (eBPF) permits sandboxed programs to execute in kernel space without requiring module loading or kernel source modification. While eBPF serves legitimate purposes in observability and security monitoring, it has been increasingly weaponised by advanced threat actors. Tools such as *Pamspy* and *TripleCross* have demonstrated eBPF-based rootkits capable of intercepting syscalls, hiding processes, and exfiltrating data within the kernel without loading a detectable module.

The `linux.ebpf` plugin enumerated all eBPF programs loaded in kernel memory at the time of acquisition. Four programs were identified and assessed as follows:

Table 4: eBPF programs identified in memory and their assessed attribution

tableheader Program Name	Type	Assessment
<code>restrict_files</code>	<code>BPF_PROG_TYPE_LSM</code>	Linux Security Module hook filesystem access restriction; legitimate system component
<code>sd_devices</code>	<code>BPF_PROG_TYPE_CGROUP_DEVICE</code>	Device cgroup policy enforcement - standard <code>systemd-udevd</code> operation
<code>sd_fw_egress</code>	<code>BPF_PROG_TYPE_CGROUP_SKB</code>	Outbound network firewall rule - <code>systemd-networkd</code>
<code>sd_fw_ingress</code>	<code>BPF_PROG_TYPE_CGROUP_SKB</code>	Inbound network packet filter - <code>systemd-networkd</code> firewall rule

```
(gamebot@gamebot)~[~/tools/volatility3]
$ python3 vol.py -f ~/tools/kali_dump.lime linux.ebpf.EBPF
Volatility 3 Framework 2.28.0
Progress: 100.00 Stacking attempts finished
Address Name Tag Type
0xc94f80211000 restrict_filesy 31efb7c56239148b BPF_PROG_TYPE_LSM
0xc94f80075000 sd_devices 4ea3934a7fabf5c3 BPF_PROG_TYPE_CGROUP_DEVICE
0xc94f800a5000 sd_fw_egress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f800bd000 sd_fw_ingress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f800c5000 sd_fw_egress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f80181000 sd_fw_ingress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f80189000 sd_devices a97c143260cd9940 BPF_PROG_TYPE_CGROUP_DEVICE
0xc94f80171000 sd_devices a97c143260cd9940 BPF_PROG_TYPE_CGROUP_DEVICE
0xc94f8056a000 sd_devices a97c143260cd9940 BPF_PROG_TYPE_CGROUP_DEVICE
0xc94f80591000 sd_devices b3333f074dcc518c BPF_PROG_TYPE_CGROUP_DEVICE
0xc94f80221000 sd_devices b1140369ce06e874 BPF_PROG_TYPE_CGROUP_DEVICE
0xc94f8062f000 sd_fw_egress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f80752000 sd_fw_ingress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f8193d000 sd_fw_egress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f81959000 sd_fw_ingress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f823a9000 sd_fw_egress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
0xc94f823c1000 sd_fw_ingress 772db7720b2728e9 BPF_PROG_TYPE_CGROUP_SKB
```

Figure 7: eBPF enumeration all four programs attributable to legitimate systemd components

4.7 Bash History Recovery linux.bash

The `linux.bash` plugin extracts command history from live bash session buffers in RAM, targeting entries that remain in memory even after `.bash_history` has been deleted from disk. This can reveal attacker-issued commands after log tampering.

The plugin completed without error but returned no results. This is consistent with the terminal session remaining open at the time of acquisition; bash flushes its in-memory command buffer to disk only on clean shell exit. Because the session was still active, prior commands were not yet accessible in recoverable memory regions.

```
(gamebot@gamebot)~[~/tools/volatility3]
$ python3 vol.py -f ~/tools/kali_dump.lime linux.bash.Bash
Volatility 3 Framework 2.28.0
Progress: 100.00 Stacking attempts finished
ID Process CommandTime Command
```

Figure 8: `linux.bash.Bash` output on `kali_dump.lime` plugin completed at 100% with no history rows returned; consistent with an active bash session at acquisition time

Note: Absence of bash history in memory does not confirm deliberate clearing. The shell was active when the dump was collected. Supplementary string carving of `kali_dump.lime` filtering for terms such as `sudo`, `insmod`, `python3`, `vol.py`, `lime`, and `mimikatz` is recommended as a complementary technique to recover residual command artefacts.

5. Incident Timeline

The following timeline was reconstructed from Volatility3 process creation timestamps, memory-resident network artefacts, EDR alert logs, and kernel banner data recovered from the memory dump.

Table 5: Reconstructed incident timeline

Time (CET)	Event
02:14	Attacker authenticates via stolen VPN credentials of employee <i>amir.khan</i> ; remote session established
02:17	Fileless PowerShell payload executed in memory using <code>-EncodedCommand</code> flag to evade detection
02:31	Process hollowing of legitimate <code>svchost.exe</code> performed; malicious shell-code injected into process memory
02:44	Mimikatz-style LSASS memory access attempted; credential dumping from <code>lsass.exe</code> process memory
03:10	Sensitive financial records staged in temporary directory; data compressed for exfiltration
03:27	EDR alert raised: active outbound connection detected to Tor exit node <code>185.220.101.47</code> on port <code>4444</code>
03:27	Network connection terminated by automated firewall rule; IR team notified via Priority 1 alert
03:45	SOC Lead loads LiME kernel module; 4.0 GB full RAM image acquired (<code>kali_dump.lime</code>)
03:55	SHA-256 hash recorded; forensic copy transferred to write-blocked analysis station
04:20	Volatility3 analysis commenced; <code>banners.Banners</code> confirms kernel; all plugins executed in sequence

The timeline reveals a methodical intrusion that operated entirely within user space. A valid VPN session was leveraged first; the attacker subsequently moved laterally using native scripting capabilities without installing new software. Detection only occurred after prolonged quiet movement across trusted system functions.

6. Findings Summary

The table below consolidates all Volatility3 plugin results and their forensic significance for the Operation Shadow Ledger investigation.

Table 6: Consolidated findings summary

Plugin	Result	Forensic Verdict
banners	Kernel 6.18.12+kali-amd64 confirmed	Dump integrity and OS baseline established
linux.pslist	60+ processes; PID 19016 flagged	Suspicious truncated name; warrants further review
linux.psscan	No unlinked processes found	Clean - no DKOM rootkit technique detected
linux.lsmmod	Standard VM hardware drivers only	Clean - no rogue or attacker-installed modules
linux.hidden_modules	No hidden modules detected	Clean - no kernel rootkit present at capture
linux.check_syscall	All 400+ handlers legitimate	Clean - no syscall hook detected
linux.check_creds	No shared credential structures	Clean - no kernel privilege escalation exploit
linux.ebpf	4 systemd eBPF programs found	Normal - all attributable to legitimate system
linux.bash	No history recovered	Inconclusive - active session at acquisition time

Key Finding: No external malware, kernel rootkit, hidden module, or syscall hook was identified. The intrusion operated entirely in user space through the abuse of legitimate tools a *living-off-the-land* approach deliberately chosen to minimise forensic visibility and evade kernel level detection.

7. Mitigation and Recommendations

7.1 Immediate Incident Response Actions

1. **Isolate the affected workstation** from the corporate network immediately and preserve its current state pending supplementary disk forensics and string carving of the RAM dump.

2. **Revoke all credentials**, VPN certificates, and SSO tokens associated with the compromised *amir.khan* account, and reset all accounts with shared credential exposure.
3. **Notify the relevant Data Protection Authority** under GDPR Article 33 if personal data was confirmed or suspected to have been exfiltrated. The 72-hour breach notification obligation applies.
4. **Initiate legal proceedings** in coordination with FinTrack's legal counsel under StGB 202a (Unauthorised Data Access) and conduct SIEM log correlation across all files accessed by the compromised account in the 30 days prior to the incident.

7.2 Technical Controls

1. **Enforce MFA on all remote access points**, including VPN. Stolen credentials alone should not provide access. This incident demonstrated they were sufficient that must change.
2. **Protect LSASS** using Windows Credential Guard on Windows endpoints and a hardened PAM configuration on Linux systems. Without this, tools such as Mimikatz can extract plaintext credentials from memory during an active session.
3. **Block outbound connections** to known Tor exit nodes and non-whitelisted external addresses at the firewall. The exfiltration channel in this incident used port 4444 to a Tor exit node standard egress filtering would have detected this earlier.
4. **Audit all Docker containers and images** on the compromised machine. Docker was active during the incident and a container escape cannot yet be ruled out. Bind mounts and runtime behaviour warrant particular attention.

8. Conclusion

The FinTrack Solutions GmbH incident was examined using Volatility3 v2.28.0 against a memory image obtained at 03:45 CET on 20 March 2026. The image was acquired using LiME on a live system and is forensically sound. All analysis was conducted exclusively on a write-protected copy; the original memory dump was kept intact and its integrity verified using SHA-256 hashing throughout the process.

Nine plugin checks covered the full range of kernel-level threats: process enumeration, hidden module detection, syscall table validation, credential structure sharing, and eBPF program analysis. Every check returned clean. The kernel was untouched no rootkits, no hidden modules, no hooked syscalls, and no evidence of privilege escalation.

That absence is itself a significant finding. The attacker never needed to engage kernel space. The entire intrusion ran in user space using tools the system already trusted: encoded PowerShell, process hollowing, Mimikatz-style LSASS credential dumping, and a

Tor-based exfiltration channel on port 4444. Seventy three minutes elapsed before any alert was triggered. This level of patience, combined with deliberate living-off-the-land techniques, is what makes this class of attack difficult to detect and difficult to attribute.

All evidence was handled in strict accordance with NIST SP 800-86 and applicable German legal frameworks, including StGB 202a-202d. The chain of custody remained intact from acquisition through analysis, and no material was altered at any stage. Should this matter proceed to legal review, the evidentiary record is sound.

The recommendations in Section 7, if implemented, would close most of the gaps exploited in this attack. MFA enforcement, LSASS protection, egress filtering, and Docker auditing are not complex changes but none were in place when it mattered.

References

- [1] Kent, K. et al. (2006) *Guide to Integrating Forensic Techniques into Incident Response*. NIST SP 800-86. Gaithersburg: NIST.
- [2] Ligh, M.H. et al. (2014) *The Art of Memory Forensics*. Indianapolis: Wiley.
- [3] Volatility Foundation (2024) *Volatility3 Framework Documentation*. Available at: <https://volatility3.readthedocs.io> [Accessed: 20 March 2026].
- [4] 504ensics Labs (2023) *LiME: Linux Memory Extractor*. Available at: <https://github.com/504ensicsLabs/LiME> [Accessed: 20 March 2026].
- [5] Volatility Foundation (2024) *dwarf2json*. Available at: <https://github.com/volatilityfoundation/dwarf2json> [Accessed: 20 March 2026].
- [6] MITRE Corporation (2024) *ATT&CK for Enterprise Linux*. Available at: <https://attack.mitre.org> [Accessed: 20 March 2026].
- [7] European Parliament (2016) *General Data Protection Regulation* (EU) 2016/679. Official Journal of the European Union.
- [8] Bundesministerium der Justiz (2023) *StGB 202a-202d*. Available at: <https://www.gesetze-im-internet.de/stgb> [Accessed: 20 March 2026].